

# Clase 2 · Laboratorio — Diseño dimensional para Saber 11

**Trabajo en parejas · 90 min.**

Al final deben tener un notebook completado con las 4 tareas resueltas y subirlo a Moodle antes de las 23:59.

**Checkpoint conjunto a los 55 min:** el profesor detiene la sala y revisamos juntos la Tarea 3 (hecho + FKs) antes de pasar al diseño del proyecto propio.

```
In [1]: import pandas as pd

df = pd.read_csv("saber11_muestra_500k.csv")
print(f"Cargados {len(df):,} registros – {df.shape[1]} columnas")
```

Cargados 500,000 registros – 22 columnas

## Tarea 1 (15 min) — dim\_colegio con clave surrogate

Construye una tabla de dimensión `dim_colegio` que contenga:

- Una clave surrogate `colegio_id` (entero secuencial).
- Los atributos: `COLE_NATURALEZA`, `COLE_JORNADA`, `COLE_CALEDARIO`, `COLE_BILINGUE`.

Requisitos:

- Sin duplicados (una fila por combinación única de los 4 atributos).
- Verifica que la clave surrogate es única.

**Respuesta:** `dim_colegio` quedó con 40 filas (las 40 combinaciones únicas que aparecen de `COLE_NATURALEZA`, `COLE_JORNADA`, `COLE_CALEDARIO` y `COLE_BILINGUE`), cada una con su `colegio_id` del 1 al 40, y la clave sí resultó única.

**Justificación:** Usé `drop_duplicates()` sobre las 4 columnas para quedarme con una fila por combinación, y saqué la clave surrogate con `dim_colegio.index + 1` para que empezara en 1 y no en 0. Al final comprobé con `assert` que `colegio_id` fuera único, como pedía el enunciado.

## Tarea 2 (15 min) — dim\_geografia con jerarquía

Construye `dim_geografia` con:

- Clave surrogate `geo_id`.
- Atributos: `COLE_DEPTO_UBICACION` (padre), `COLE_MCPIO_UBICACION` (hijo).

Requisitos:

- Una fila por par (departamento, municipio).
- Documenta en Markdown por qué la jerarquía es útil para el análisis.

**Respuesta:** `dim_geografia` quedó con 10 filas, una por cada departamento. En este dataset de muestra cada departamento solo tiene un municipio registrado (parece que usaron la capital de cada uno), así que en este caso no hay municipios "de más" dentro de cada departamento, pero la estructura queda lista por si el dataset tuviera más.

**Justificación:** La jerarquía `depto` -> `municipio` sirve porque así después puedo "subir" de nivel (drill-up) y sacar promedios por departamento sin tocar la tabla de hechos, simplemente agrupando por `geo_id` y trayendo el departamento desde `dim_geografia`. También sirve para filtrar rápido: si el profesor pregunta por un departamento específico, ya tengo esa columna lista en la dimensión en vez de tenerla repetida 500,000 veces en la tabla de hechos.

## Tarea 3 (25 min) — hecho\_resultados

Construye la tabla de hechos `hecho_resultados` con:

- FKs a: `dim_colegio` (`colegio_id`), `dim_geografia` (`geo_id`), `dim_tiempo` (`tiempo_id` — bosqueja también esta dimensión).
- Medidas: `PUNT_LECTURA_CRITICA`, `PUNT_MATEMATICAS`, `PUNT_C_NATURALES`, `PUNT_SOCIALES_CIUDADANAS`, `PUNT_INGLES`, `PUNT_GLOBAL`.

Requisitos:

- La granularidad del hecho es "una fila por estudiante-periodo".
- Después de hacer los joins con las dimensiones, la tabla de hechos NO debe perder filas (comparar `len(hecho_resultados)` con `len(df)` original).

**Respuesta:** `hecho_resultados` quedó con 500,000 filas, exactamente las mismas que el `df` original, o sea que no se perdió ninguna fila en los joins. Tampoco salió ningún nulo en las llaves foráneas (`colegio_id`, `geo_id`, `tiempo_id`).

**Justificación:** Hice los merges con `how="left"` empezando siempre desde `df` (no desde las dimensiones), porque así me aseguro de conservar todas las filas originales aunque alguna combinación no calzara con la dimensión. En este caso no hubo nulos porque las 3 dimensiones se construyeron directamente a partir de las mismas columnas del `df`, entonces toda combinación que aparece en el hecho ya existía en su dimensión correspondiente. Si estuviera cargando una dimensión desde otra fuente (por ejemplo un

catálogo de colegios aparte), ahí sí podría pasar que un colegio del hecho no exista en la dimensión y me quedara colegio\_id en null — en ese caso tocaría revisar si es un dato mal escrito o si falta agregarlo a la dimensión.

## Checkpoint del profesor (10 min)

Detengan aquí. El profesor revisa la Tarea 3 con toda la sala: cómo evitar perder filas al hacer joins, qué hacer si aparecen nulos en las FKs.